

2.3 사이킷런의 기반 프레임워크 익히기

목차

1. Estimator 이해 및 `fit()`, `predict()` 메서드
 1. Estimator란 무엇인가
 2. Classifier와 Regressor
 3. 비지도학습에서의 `fit()`과 `transform()`
2. 사이킷런의 주요 모듈
3. 내장 예제 데이터 세트
 1. 데이터 세트의 유형
 2. Bunch 객체의 타입 확인
 3. Bunch 객체의 키 확인
 4. 개별 데이터 상세 확인
4. 핵심 요약 및 머신러닝 활용 포인트

1. Estimator 이해 및 fit(), predict() 메서드

1.1 Estimator란 무엇인가

사이킷런은 매우 일관되고 단순한 API 체계를 가지고 있습니다. 그 중심에 있는 개념이 **Estimator(추정기)**입니다. 사이킷런에서 학습과 예측을 수행하는 모든 알고리즘 클래스를 통칭해 Estimator라고 부릅니다.

Estimator의 핵심은 어떤 알고리즘이든 동일한 두 개의 메서드를 가진다는 점입니다.

메서드	역할	인자
<code>fit()</code>	학습. 데이터에서 규칙(모델 파라미터)을 찾아 객체 내부에 저장합니다.	학습용 피쳐 X, 학습용 레이블 y
<code>predict()</code>	예측. 학습된 규칙을 새 데이터에 적용해 결과를 반환합니다.	예측할 피쳐 X만 (레이블 없음)

■ 왜 이 설계가 강력한가

알고리즘을 바꾸려면 객체 생성 한 줄만 바꾸면 됩니다. 나머지 `fit( )`, `predict( )`, 평가 코드는 전혀 수정할 필요가 없습니다. 덕분에 여러 알고리즘을 빠르게 비교하고, `GridSearchCV` 나 `cross_val_score` 같은 범용 도구가 모든 알고리즘에 그대로 적용될 수 있습니다.

1.2 Classifier와 Regressor

지도학습의 두 축인 분류와 회귀에 따라 Estimator는 다음과 같이 나뉩니다.

구분	설명	대표 클래스
Classifier (분류)	예측 대상이 이산적인 클래스(품종, 생존 여부 등)	<code>DecisionTreeClassifier</code> , <code>RandomForestClassifier</code> , <code>LogisticRegression</code> , <code>GradientBoostingClassifier</code>
Regressor (회귀)	예측 대상이 연속적인 숫자값(가격, 온도 등)	<code>LinearRegression</code> , <code>Ridge</code> , <code>Lasso</code> , <code>RandomForestRegressor</code>

클래스 이름 끝에 **Classifier**가 붙으면 분류, **Regressor**가 붙으면 회귀라는 명명 규칙이 일관되게 적용되므로, 이름만 보고도 용도를 알 수 있습니다. 이 둘을 합쳐 **Estimator**라고 부릅니다.

1.3 비지도학습에서의 fit()과 transform()

차원 축소, 클러스터링, 피쳐 추출, 그리고 데이터 전처리 클래스들도 동일하게 `fit( )` 과 `transform( )` 을 사용합니다. 다만 의미가 조금 다릅니다.

메서드	지도학습에서	비지도학습·전처리에서
<code>fit()</code>	데이터를 학습해 예측 모델을 만듦	입력 데이터의 구조·통계(평균, 표준편차, 최솟값 등)를 파악만 함. 실제 변환은 하지 않음
<code>transform()</code>	—	fit에서 파악한 정보를 바탕으로 실제 데이터를 변환
<code>fit_transform()</code>	—	두 작업을 한 번에 수행

⚠ `fit_transform()` 사용 시 주의

테스트 데이터에는 절대 `fit()` 이나 `fit_transform()` 을 쓰면 안 됩니다. 학습 데이터로 `fit()` 한 기준을 테스트 데이터에도 동일하게 적용해야 하므로, 테스트 데이터에는 `transform()` 만 호출합니다. 이 내용은 2.5장에서 실습으로 확인합니다.

2. 사이킷런의 주요 모듈

사이킷런은 기능 영역별로 모듈이 잘 정리되어 있습니다. 자주 사용하는 모듈은 다음과 같습니다.

분류	모듈명	설명
예제 데이터	<code>sklearn.datasets</code>	사이킷런에 내장되어 예제로 제공하는 데이터 세트 (<code>load_iris</code> , <code>load_digits</code> 등)
피처 처리	<code>sklearn.preprocessing</code>	데이터 전처리에 필요한 가공 기능 (인코딩, 정규화, 스케일링)
	<code>sklearn.feature_selection</code>	영향력이 큰 피처를 우선순위로 선택하는 기능
	<code>sklearn.feature_extraction</code>	텍스트·이미지 데이터에서 벡터화된 피처를 추출
차원 축소	<code>sklearn.decomposition</code>	PCA, NMF, Truncated SVD 등 차원 축소 알고리즘
모델 선택	<code>sklearn.model_selection</code>	학습/검증/테스트 데이터 분리, 교차 검증, 하이퍼 파라미터 튜닝(GridSearchCV)
평가	<code>sklearn.metrics</code>	정확도, 정밀도, 재현율, ROC-AUC, RMSE 등 성능 측정 지표
ML 알고리즘	<code>sklearn.tree</code>	의사 결정 트리 알고리즘
	<code>sklearn.ensemble</code>	랜덤 포레스트, 그래디언트 부스팅, 에이다 부스트 등 앙상블
	<code>sklearn.linear_model</code>	선형 회귀, 릿지, 라쏘, 로지스틱 회귀, SGD
	<code>sklearn.naive_bayes</code>	가우시안 NB, 다항 분포 NB 등 나이브 베이즈
	<code>sklearn.cluster</code>	K-Means, DBSCAN, 평균 이동 등 비지도 클러스터링
유틸리티	<code>sklearn.pipeline</code>	피처 처리와 학습·예측을 함께 묶어 실행하는 유틸리티

■ 일반적인 머신러닝 코드 작성 순서

① `datasets` 또는 판다스로 데이터 로딩 → ② `preprocessing` 으로 가공 → ③ `model_selection` 으로 데이터 분리 → ④ 알고리즘 모듈로 학습·예측 → ⑤ `metrics` 로 평가. 필요하다면 다시 ②로 돌아가 반복 개선합니다.

3. 내장 예제 데이터 세트

3.1 데이터 세트의 유형

`sklearn.datasets` 모듈은 크게 두 가지 방식으로 데이터를 제공합니다.

접두어	의미	예시
<code>load_</code>	사이킷런에 내장되어 함께 설치된 소규모 데이터. 즉시 사용 가능	<code>load_iris()</code> , <code>load_breast_cancer()</code> , <code>load_digits()</code> , <code>load_diabetes()</code>

fetch_	용량이 커서 인터넷에서 내려받아 사용하는 데이터	fetch_20newsgroups(), fetch_olivetti_faces(), fetch_covtype()
make_	분류·회귀용 가상 데이터를 생성	make_classification(), make_blobs()

3.2 Bunch 객체의 타입 확인

```
from sklearn.datasets import load_iris

iris_data = load_iris()
print(type(iris_data))
```

줄	코드	상세 설명
1	from sklearn.datasets import load_iris	붓꽃 데이터 로딩 함수를 임포트합니다.
3	iris_data = load_iris()	데이터를 로딩합니다. 인자 없이 호출하면 기본 형태로 반환됩니다.
4	print(type(iris_data))	결과는 <code><class 'sklearn.utils._bunch.Bunch'></code> 입니다. Bunch 는 파이썬 딕셔너리를 상속한 자료형으로, 딕셔너리처럼 <code>iris_data['data']</code> 로도, 속성처럼 <code>iris_data.data</code> 로도 둘 다 접근할 수 있는 것이 특징입니다.

▶ 실행 결과

```
<class 'sklearn.utils._bunch.Bunch'>
```

⚠ 버전 차이

사이킷런 0.22 이전 버전에서는 `sklearn.utils.Bunch` 로 표시되었습니다. 내부 모듈 경로만 `sklearn.utils._bunch.Bunch` 로 바뀌었을 뿐 사용 방법은 완전히 동일합니다.

3.3 Bunch 객체의 키 확인

```
keys = iris_data.keys()
print('붓꽃 데이터 세트의 키들:', keys)
```

줄	코드	상세 설명
1	keys = iris_data.keys()	Bunch가 딕셔너리를 상속했으므로 <code>keys()</code> 메서드를 그대로 쓸 수 있습니다. 어떤 정보가 들어 있는지 목록을 확인합니다.
2	print('붓꽃 데이터 세트의 키들:', keys)	8개의 키가 출력됩니다. 각 키의 의미는 아래 표를 참고하세요.

▶ 실행 결과

```
붓꽃 데이터 세트의 키들: dict_keys(['data', 'target', 'frame', 'target_names',  
'DESCR', 'feature_names', 'filename', 'data_module'])
```

키	내용
data	피쳐 데이터 세트. numpy ndarray. 머신러닝의 입력 X가 됩니다.
target	레이블(정답) 데이터. 분류에서는 클래스 값, 회귀에서는 숫자 결과값. ndarray.
target_names	개별 레이블 숫자에 대응하는 이름. ndarray 또는 리스트.

feature_names	피쳐의 이름. 리스트.
DESCR	데이터 세트에 대한 설명 문서와 각 피쳐의 통계 요약. 문자열.
frame	as_frame=True 로 로딩했을 때의 DataFrame. 기본값은 None.
filename , data_module	원본 CSV 파일 경로와 모듈 위치. 실습에서는 거의 사용하지 않습니다.

3.4 키별 데이터 상세 확인

```
print('\n feature_names 의 type:', type(iris_data.feature_names))
print(' feature_names 의 shape:', len(iris_data.feature_names))
print(iris_data.feature_names)

print('\n target_names 의 type:', type(iris_data.target_names))
print(' feature_names 의 shape:', len(iris_data.target_names))
print(iris_data.target_names)

print('\n data 의 type:', type(iris_data.data))
print(' data 의 shape:', iris_data.data.shape)
print(iris_data['data'])

print('\n target 의 type:', type(iris_data.target))
print(' target 의 shape:', iris_data.target.shape)
print(iris_data.target)
```

줄	코드	상세 설명
1	type(iris_data.feature_names)	<class 'list'> → 피쳐 이름은 파이썬 리스트입니다. ndarray가 아니므로 shape 대신 len() 을 씁니다.
2	len(iris_data.feature_names)	4 → 피쳐가 4개임을 의미합니다.
3	print(iris_data.feature_names)	꽃받침 길이/너비, 꽃잎 길이/너비 4개 이름이 단위(cm)와 함께 출력됩니다.
5	type(iris_data.target_names)	numpy.ndarray → 레이블 이름은 ndarray입니다. feature_names와 타입이 다르다는 점에 주의하세요.
6	len(iris_data.target_names)	3 → 품종이 3종류입니다. (주석의 'feature_names'는 원본 코드의 오타이며 실제로는 target_names의 길이입니다.)
7	print(iris_data.target_names)	setosa, versicolor, virginica. 이 순서가 곧 레이블 0, 1, 2에 대응합니다.
9	type(iris_data.data)	numpy.ndarray → 피쳐 데이터는 ndarray입니다.
10	iris_data.data.shape	(150, 4) → 150개의 샘플(로우), 4개의 피쳐(컬럼)를 가진 2차원 배열입니다.
11	print(iris_data['data'])	대괄호 방식으로 접근한 예입니다. iris_data.data 와 완전히 동일한 결과를 반환합니다 — Bunch가 두 방식을 모두 지원함을 보여 줍니다. 150행 전체가 출력되므로 지면상 앞부분만 실었습니다.
13	type(iris_data.target)	numpy.ndarray .
14	iris_data.target.shape	(150,) → 150개 원소의 1차원 배열입니다. 피쳐가 2차원인 것과 대비됩니다.
15	print(iris_data.target)	0이 50개, 1이 50개, 2가 50개 순서대로 정렬되어 있습니다. 이 정렬 상태는 교차 검증 시 문제를 일으키며, 2.4장의 Stratified K 폴드에서 다룹니다.

▶ 실행 결과

4.2 Bunch 키 접근 요약

코드	반환 타입	shape / 길이	용도
<code>iris.data</code>	ndarray	(150, 4)	학습 입력 X
<code>iris.target</code>	ndarray	(150,)	학습 정답 y
<code>iris.feature_names</code>	list	4	DataFrame 컬럼명
<code>iris.target_names</code>	ndarray	3	예측 결과 해석
<code>iris.DESCR</code>	str	—	데이터 설명 확인

4.3 머신러닝 활용 포인트

- **알고리즘 교체가 한 줄**: Estimator 인터페이스가 통일되어 있으므로, 여러 알고리즘의 성능을 비교하는 코드를 반복문 한 번으로 작성할 수 있습니다.
- **Pipeline과의 연결**: `fit / transform` 규약을 지키기 때문에 전처리와 모델을 `Pipeline`으로 묶어 **데이터 누수(data leakage) 없이** 한 번에 처리할 수 있습니다.
- **예제 데이터의 역할**: 내장 데이터는 크기가 작고 결측치가 없어 **알고리즘 자체의 동작을 익히는 데** 적합합니다. 실제 데이터의 지저분함은 2.6장 타이타닉 실습에서 경험합니다.
- **DESCR 활용**: 낯선 예제 데이터를 만나면 `print(data.DESCR)`로 **피쳐 의미와 통계 요약**을 먼저 읽어 보는 습관이 좋습니다.

⚠ 안정화 버전 참고 (scikit-learn 1.7.x 기준)

- Bunch의 클래스 경로가 `sklearn.utils._bunch.Bunch`로 표시됩니다(과거 `sklearn.utils.Bunch`). 사용법은 동일합니다.
- 키 목록에 `data_module`이 추가되어 있습니다. 구버전에는 없던 항목이지만 실습에는 영향이 없습니다.
- `load_boston()`은 윤리적 문제로 **1.2 버전에서 완전히 제거**되었습니다. 회귀 예제로는 `fetch_california_housing()`을 사용하세요.